

Network Traffic Analysis Using Wireshark

A Comprehensive Network Protocol & Security Analysis

Submitted By	Priyanka Kumari
Course	Cybersecurity / Computer Science Engineering
Tool Used	Wireshark v4.x (Network Protocol Analyzer)
Platform	Windows OS Wi-Fi Interface (802.11)
Date	June 07, 2026

ABSTRACT

This report presents a hands-on analysis of real-time network traffic captured using Wireshark on a personal Wi-Fi interface. Multiple protocols — DNS, TCP, HTTP, HTTPS/TLS, and ARP — were captured, filtered, and analyzed with actual screenshots as evidence. Network anomalies including TCP Retransmissions, Duplicate ACKs, and Spurious Retransmissions were identified and interpreted from a cybersecurity perspective.

DNS

TCP

HTTP

HTTPS/TLS

ARP

Packet
Analysis

Network
Security

1. Introduction

In today's interconnected world, **network traffic analysis** is one of the most essential skills for any cybersecurity professional. Every packet that travels across a network tells a story — who communicated with whom, what protocol was used, and whether the data was protected. This project captures that story in real time.

Using **Wireshark** — the world's leading open-source packet analyzer — live traffic was captured on a personal Wi-Fi interface. The captured data was then filtered and analyzed across five major protocols: **DNS, TCP, HTTP, HTTPS/TLS, and ARP**. All screenshots in this report are actual captures from the live session, serving as practical evidence.

2. Tools & Environment

Tool	Details	Purpose
Wireshark	v4.x — Windows	Live packet capture & protocol analysis
Wi-Fi Interface	802.11 (AzureWaveTec NIC)	Capture medium — all traffic on this interface
Chrome Browser	Latest	Generate HTTP/HTTPS/DNS traffic for analysis
Display Filters	BPF Syntax	dns http tcp arp tcp.stream eq 1

3. Methodology

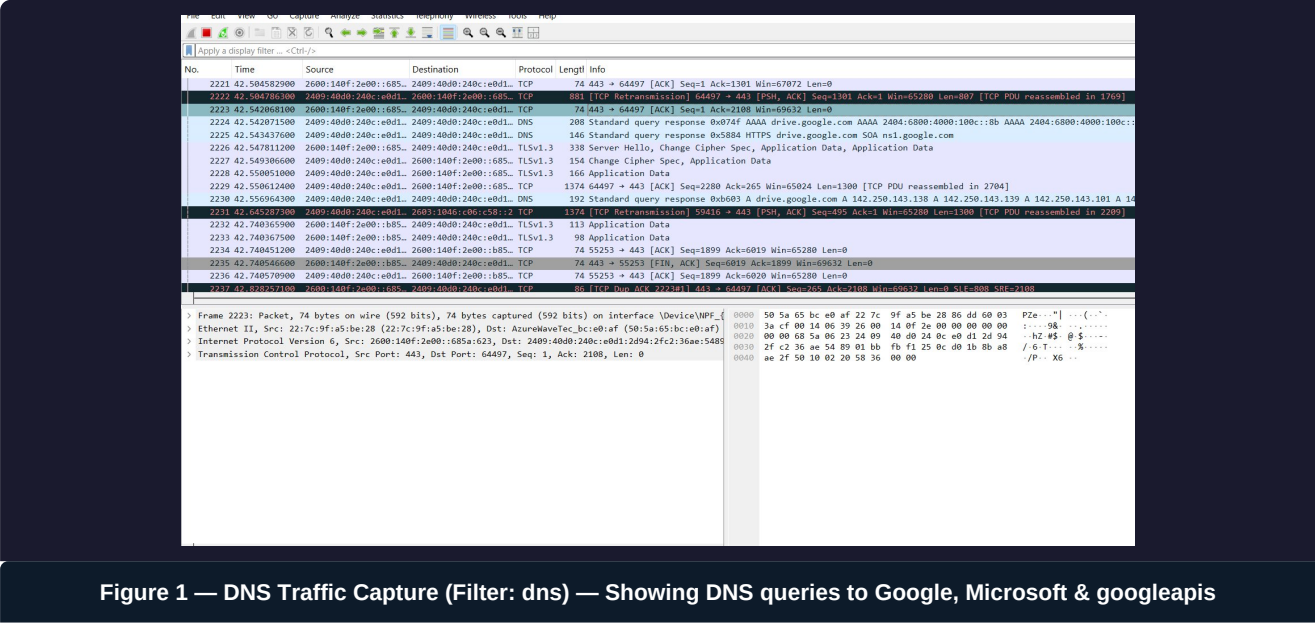
1	Wireshark installed and launched on Windows system with WinPcap/Npcap driver
2	Wi-Fi interface (AzureWaveTec) selected — this captures all wireless traffic
3	Capture started using the blue shark-fin button; packets began recording instantly
4	Multiple websites opened in Chrome to generate diverse protocol traffic
5	Capture stopped after sufficient packets collected (94,000+ packets recorded)
6	Display filters applied: dns, http, tcp, arp, tcp.stream eq 1
7	Individual packets analyzed — headers, flags, payloads, anomalies inspected
8	Screenshots taken as evidence and documented with security observations

4. Protocol Analysis & Observations

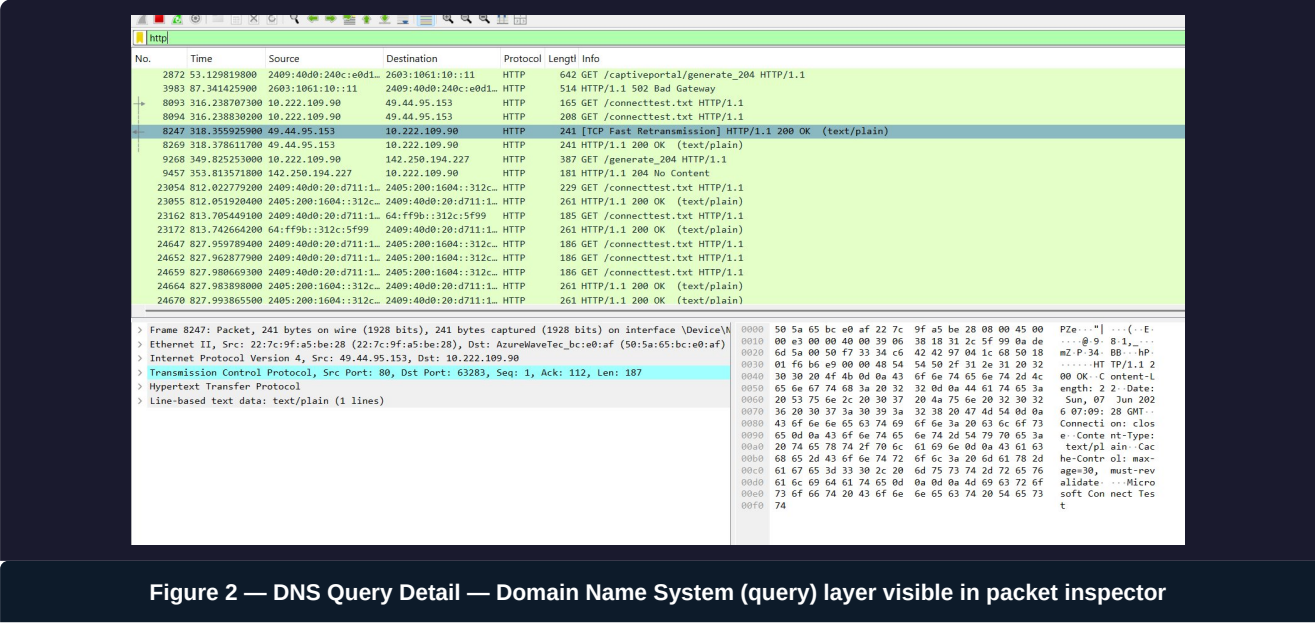
4.1 DNS — Domain Name System Analysis

DNS is the internet's phonebook — every website visit starts with a DNS query to resolve domain names to IP addresses. The filter **dns** was applied in Wireshark to isolate all DNS packets. DNS runs on **UDP port 53** and transmits data in plain text, making it fully visible to any network observer.

Filter Applied: dns



Filter Applied: dns | Packet detail pane expanded



Domain Queried	Query Type	Destination	Security Note
www.google.com	A / AAAA	10.222.109.89	Normal — standard lookup
googleapis.com	AAAA	10.222.109.89	Google API — normal
functional.events.data.microsoft.com	CNAME	10.222.109.89	Telemetry — monitor

substrate.office.com

A / AAAA

10.222.109.89

MS Office telemetry

Observation: DNS traffic is unencrypted. An attacker on the same Wi-Fi can monitor every DNS query and build a complete profile of browsing activity. The capture shows queries to both Google and Microsoft telemetry domains — normal for a Windows system, but worth monitoring.

Security Risk:

DNS Spoofing / Cache Poisoning: Without DNSSEC, attackers can forge DNS replies and redirect users to malicious servers. Mitigation: Use DNS-over-HTTPS (DoH) or DNS-over-TLS.

4.2 HTTP — Unencrypted Traffic Analysis

HTTP (HyperText Transfer Protocol) sends all data in **plain text**. The filter **http** was applied to isolate HTTP packets. As seen in the captures, full request/response headers are readable — including server identity, content type, and connection details. This is a serious security vulnerability.

Filter Applied: http

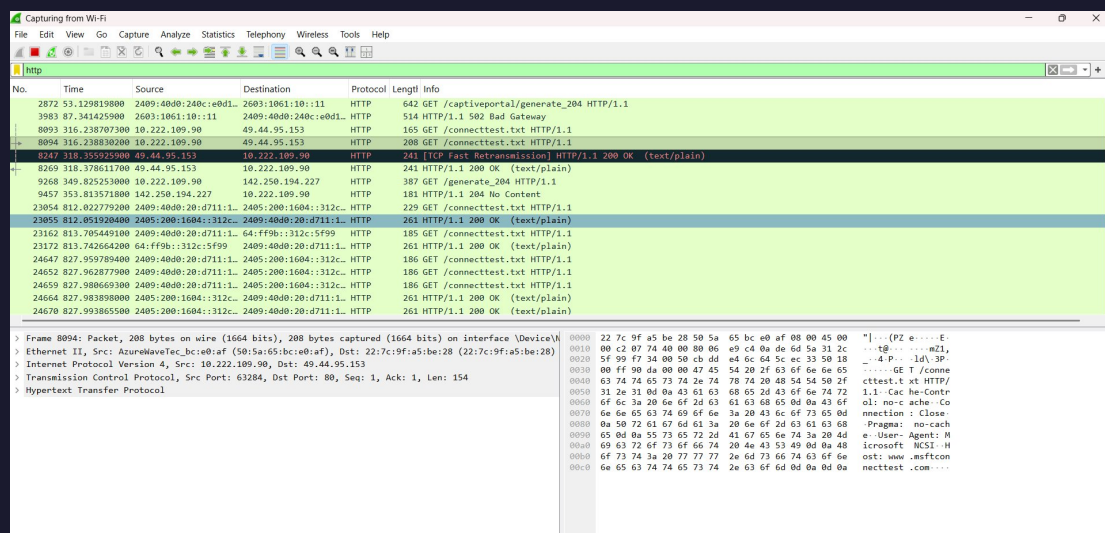


Figure 3 — HTTP Traffic Capture (Filter: http) — GET requests and 200 OK responses visible in plaintext

Filter Applied: http | Frame 8094 expanded

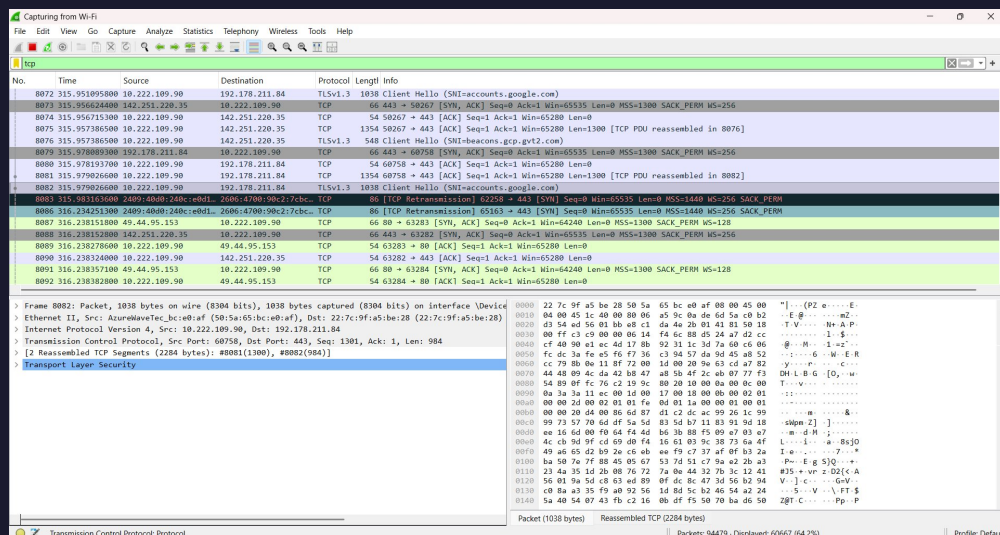


Figure 4 — HTTP Packet Detail — Full headers readable: Host, User-Agent, Cache-Control, Connection

Observed Element	Value	Risk
Request Method	GET /connecttest.txt HTTP/1.1	Low
Response	HTTP/1.1 200 OK (text/plain)	Low
Host Header	www.msftconnecttest.com	Low
Data Visibility	Complete plain text — all headers readable	CRITICAL
Fast Retransmission	[TCP Fast Retransmission] on HTTP response	Medium

Observation: HTTP packets show complete request headers including User-Agent (Microsoft NCSI), Cache-Control settings, and connection type — all in plain text. Anyone sniffing the network with Wireshark can read everything.

Security Risk: Session Hijacking / Credential Theft: On HTTP, session cookies and login credentials are transmitted in plaintext. A network attacker can capture and replay them instantly. All modern sites must use HTTPS.

4.3 TCP & Network Anomalies — Connection Layer Analysis

TCP (Transmission Control Protocol) is the transport layer backbone. Wireshark color-codes TCP anomalies automatically — **red for retransmissions** (packet loss) and **yellow for duplicate ACKs** (out-of-order delivery). The filter **tcp.stream eq 1** was used to follow a specific TCP conversation.

Filter Applied: **tcp | tcp.stream eq 1**

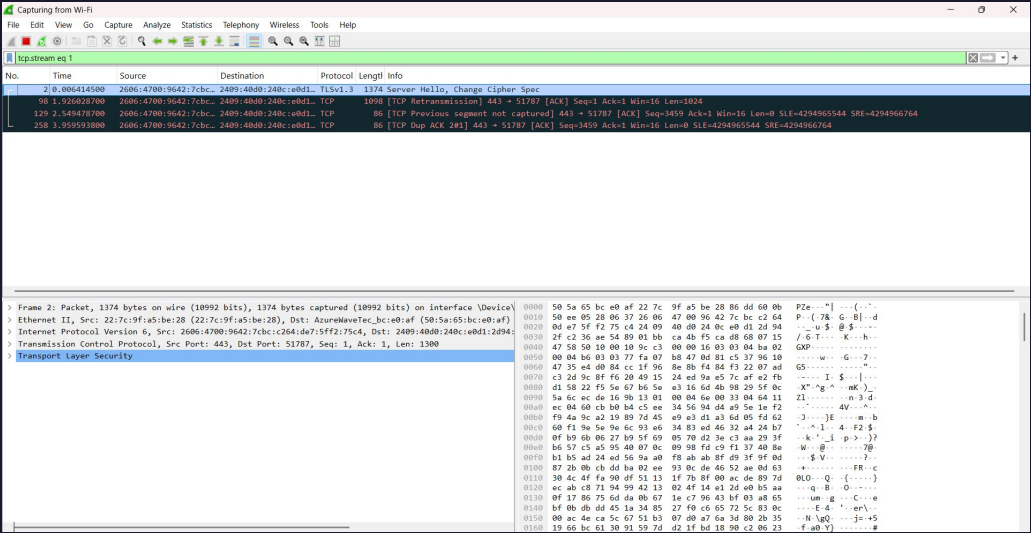


Figure 5 — TCP Traffic with Anomalies — Red rows = TCP Retransmissions, TLSv1.3 also visible

Filter Applied: No filter – full capture view

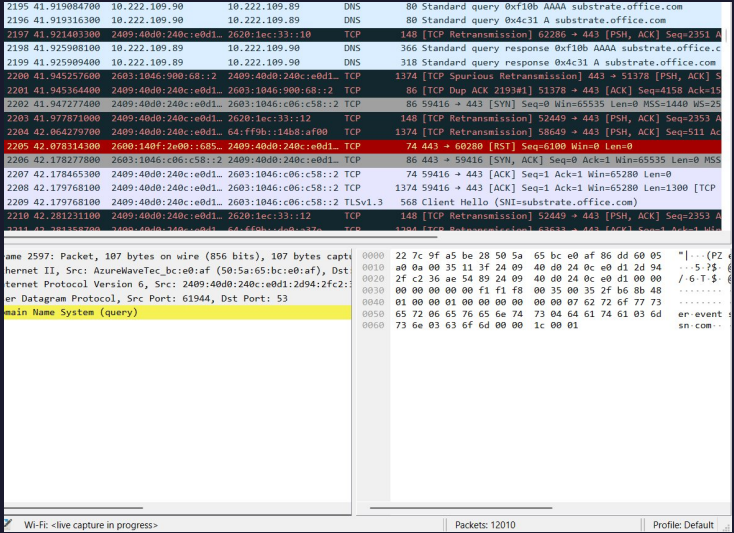


Figure 6 — Mixed Traffic View — DNS, TCP, TLSv1.3 packets visible including Duplicate ACKs and RST flags

Anomaly	Wireshark Color	Root Cause	Security Implication
TCP Retransmission	Red	Packet lost — resent	Network flooding / DoS indicator
Duplicate ACK	Yellow	Out-of-order packets	Possible packet injection
Spurious Retransmission	Red	ACK before retransmit	Network timing issue
Previous Segment Not Captured	Yellow	Capture mid-session	Incomplete forensic data
TCP RST	Black/Red	Abrupt connection kill	Port scan / firewall block
TLSv1.3 Client Hello	None	TLS handshake begin	Encrypted channel starting

Observation: The capture shows 94,479 total packets with 60,667 displayed (64.2%) after filtering. Multiple TCP retransmissions and duplicate ACKs indicate network congestion or packet loss. TCP RST packets were also observed — these can indicate port scanning activity or abrupt connection terminations by a firewall.

4.4 HTTPS / TLS 1.3 — Encrypted Communication Analysis

HTTPS uses TLS (Transport Layer Security) to encrypt all communication. Even though packets are visible in Wireshark, the payload is completely unreadable without the server's private key. The capture shows **TLSv1.3** — the most secure version — with Client Hello packets initiating encrypted sessions.

Filter Applied: tcp.stream eq 1 | TLSv1.3 handshake visible

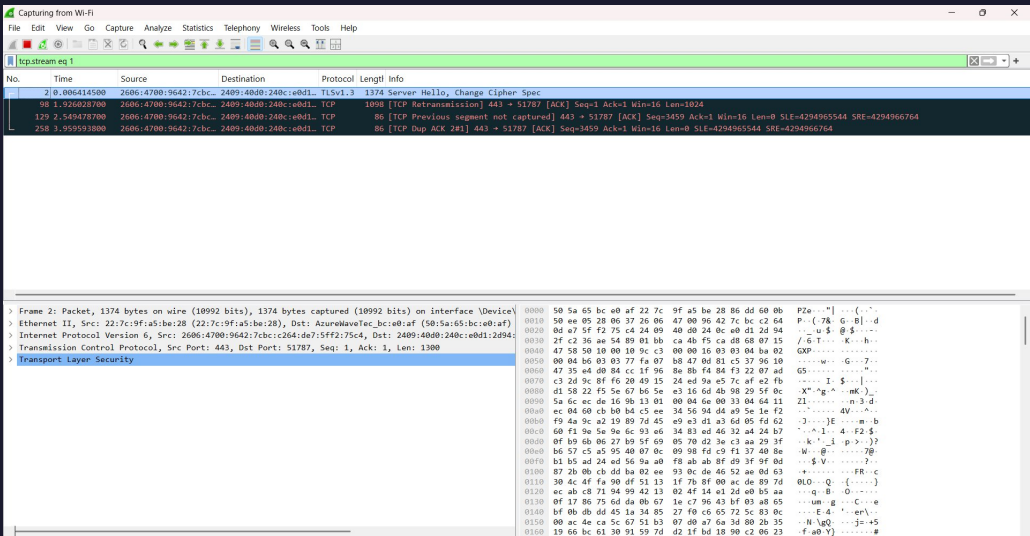


Figure 7 — TLSv1.3 Packets — Client Hello with SNI (Server Name Indication) visible, payload encrypted

TLS Step	Observed	What It Means
Client Hello	TLSv1.3 Client Hello (SNI=accounts.google.com)	Client starts handshake, sends supported ciphers
Server Hello	Server Hello, Change Cipher Spec	Server picks cipher, sends certificate
Application Data	Encrypted binary payload	All data AES-encrypted — unreadable
TCP ACK	443 → port ACK	Port 443 confirms HTTPS connection

Observation: TLSv1.3 provides Perfect Forward Secrecy (PFS) — even if the server key is stolen later, past sessions cannot be decrypted. The SNI field in Client Hello does reveal which domain is being accessed (e.g., accounts.google.com) even in HTTPS traffic.

4.5 ARP — Address Resolution Protocol Analysis

ARP maps IP addresses to MAC addresses within the local network. The filter **arp** was applied to isolate ARP traffic. ARP has no authentication — making it the primary target for **ARP Spoofing** attacks, a classic Man-in-the-Middle technique. The NIC identified in capture is **AzureWaveTec** (MAC: 22:7c:9f:a5:be:28).

Filter Applied: arp

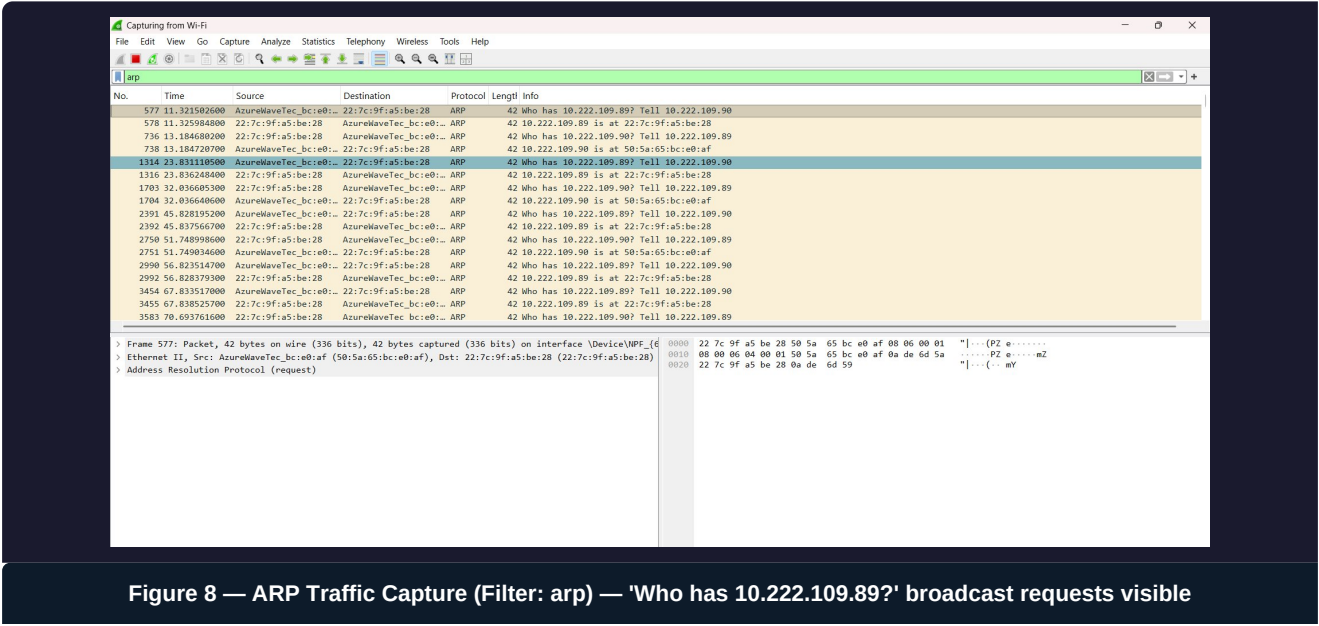


Figure 8 — ARP Traffic Capture (Filter: arp) — 'Who has 10.222.109.89?' broadcast requests visible

Observation: ARP broadcasts 'Who has 10.222.109.89? Tell 10.222.109.90' repeatedly. This is normal ARP behavior for local network communication. However, if an attacker sends unsolicited ARP replies claiming their MAC for the gateway IP, all traffic gets intercepted — this is ARP Poisoning.

■ Security Risk:

ARP Spoofing / Man-in-the-Middle: An attacker runs 'arp spoof -i eth0 -t victim gateway' to redirect all victim traffic through attacker's machine. Solution: Dynamic ARP Inspection (DAI) on managed switches, or use VPN.

5. Security Risk Assessment Summary

Protocol	Port	Encrypted?	Risk Level	Key Threat	Fix
HTTP	80	No	CRITICAL	Data theft / MITM	Use HTTPS always
DNS	53	No	HIGH	DNS Spoofing	Use DoH / DoT
ARP	L2	No	HIGH	ARP Poisoning	DAI on switches
TCP	Any	No	MEDIUM	SYN Flood / RST	Firewall + rate limit
HTTPS	443	Yes	LOW	Expired cert / SNI	Enforce HSTS

6. Conclusion

This lab successfully demonstrated real-world network traffic analysis using Wireshark. Through hands-on packet capture and analysis, the following key cybersecurity lessons were validated with actual captured evidence:

DNS	Every website visit generates DNS queries in plaintext — fully visible on the network
HTTP	HTTP traffic is completely unencrypted — server responses, headers, and data are readable
HTTPS	TLSv1.3 effectively protects data; payload is unreadable even in Wireshark
TCP Anomalies	Retransmissions and Duplicate ACKs indicate network stress and possible attacks
ARP	Local network communication is unauthenticated and vulnerable to spoofing attacks
Wireshark	Color coding and display filters make protocol analysis fast and efficient

*This project reinforced that **network visibility is the foundation of cybersecurity**. An analyst who can read and interpret network traffic can detect intrusions, investigate security incidents, and identify vulnerabilities before attackers exploit them. Wireshark is an indispensable tool in every cybersecurity professional's toolkit.*

7. Future Scope

Malware Traffic Analysis	Capture and analyze C2 (Command & Control) callback patterns using Wireshark + Suricata IDS
Kali Linux + tshark	Use headless tshark for automated captures; NetworkMiner for file extraction from pcaps
SSL/TLS Decryption	Configure browser SSL key logging to decrypt and inspect HTTPS payloads in Wireshark
Wi-Fi Attack Analysis	Monitor mode capture of WPA2 handshakes, deauthentication attacks (802.11 management frames)
PCAP Forensics	Analyze saved .pcap files from CTF challenges and real incident response scenarios

8. Ethical Consideration

All traffic captured in this project was generated on a personal device on a personal network. No third-party systems were targeted. This project was conducted purely for **educational purposes** in compliance with academic ethics guidelines.

9. References

- [1] Wireshark Foundation. (2024). Wireshark User's Guide. <https://www.wireshark.org/docs/>
- [2] Sanders, C. & Smith, J. (2017). Applied Network Security Monitoring. No Starch Press.
- [3] Stallings, W. (2022). Network Security Essentials, 6th Ed. Pearson.
- [4] RFC 793 — Transmission Control Protocol. IETF.
- [5] RFC 1035 — Domain Names Implementation. IETF.
- [6] RFC 8446 — TLS Protocol Version 1.3. IETF.
- [7] OWASP. (2024). Transport Layer Security Cheat Sheet. cheatsheetseries.owasp.org